



Universidade de Aveiro

90338-TS

Testes de Software

90338-TS: Testes de Software

**Projeto: Implementação de Testes de Integração
para uma API de Biblioteca**

Mestrado em Informática Aplicada

Ano: 1º ano

Semestre: 2º semestre

Disciplina: 90337-SAWM - Serviços para Aplicações Web e Móveis

Alunos:

134847 - Miguel Pinto Ribeiro Gonçalves



Universidade de Aveiro

90338-TS

Testes de Software

Índice



1. Breve apresentação / descrição do objeto de teste

O objeto de teste deste projeto é a API **Library**, uma aplicação REST em desenvolvimento cujo propósito é apoiar a gestão de uma biblioteca pública. A API disponibiliza operações relacionadas com três entidades fundamentais: **Book**, **Member** e **Reservation**, permitindo a criação, consulta, atualização e, em alguns casos, remoção de registos associados a cada uma destas áreas.

A aplicação é distribuída em formato *jar* e pode ser executada através do comando:

Código

```
java -jar library.jar
```

Após o arranque, a API fica acessível em <http://localhost:8080/>, sendo a documentação Swagger disponibilizada em <http://localhost:8080/swagger-ui/index.html>. A base de dados utilizada é **H2**, criada automaticamente numa pasta *tmp*, podendo esta ser removida para reiniciar o estado da aplicação.

Durante a análise inicial e ao longo da execução dos testes, foram identificadas diversas inconsistências entre o comportamento real da API e a documentação fornecida. Entre estas destacam-se validações incompletas, códigos de estado incoerentes e a ausência de determinados endpoints. Estas observações reforçaram a necessidade de uma abordagem sistemática ao processo de teste, conforme recomendado no enunciado do projeto.

2. Objetivo dos testes

O principal objetivo deste trabalho consiste em aplicar os conhecimentos adquiridos na unidade curricular de Testes de Software, validando o comportamento da API *Library* através de testes de integração. Pretendeu-se avaliar não apenas o funcionamento nominal dos endpoints, mas também a sua robustez perante dados inválidos, cenários negativos e situações de fronteira.

Assim, os testes foram concebidos para:

- Verificar o correto funcionamento das operações documentadas.
- Avaliar o comportamento da API em cenários positivos e negativos.



- Confirmar a conformidade dos códigos de estado HTTP devolvidos.
- Validar regras de negócio, incluindo restrições de unicidade e integridade.
- Analisar o comportamento da API perante reservas ativas.
- Garantir a repetibilidade dos testes através de uma estratégia de limpeza adequada.

Este conjunto de objetivos está alinhado com o enunciado, que destaca a importância de aplicar metodologias, ferramentas e boas práticas de teste ao longo de todo o processo.

3. Estratégia de testes

A estratégia de testes adotada baseou-se numa abordagem **black-box**, utilizando exclusivamente a documentação Swagger como referência para a definição dos cenários. Esta opção permitiu avaliar a API do ponto de vista do utilizador final, sem dependência da implementação interna.

Os testes foram automatizados utilizando **Java**, **JUnit 5**, **Gradle** e **RestAssured**, garantindo consistência, repetibilidade e facilidade de execução. A organização dos testes seguiu uma estrutura modular, separando cenários positivos e negativos para cada entidade:

- api.positivos.BookTest
- api.negativos.BookTest
- api.positivos.MemberTest
- api.negativos.MemberTest
- api.positivos.ReservaTest
- api.negativos.ReservaTest

Foram utilizados métodos das famílias **@BeforeAll**, **@AfterAll**, **@BeforeEach** e **@AfterEach** para preparar o ambiente antes de cada teste e garantir a limpeza após a sua execução. Para evitar colisões de dados, foram gerados valores únicos (ISBN, email, NIF) sempre que necessário.

A ordem de execução dos testes foi controlada através de **@Order** e do ficheiro junit-platform.properties, assegurando que determinados cenários dependentes fossem executados na sequência correta.



4. Ambiente de teste

O ambiente de teste foi constituído por:

- **Java 25** para a execução da API.
- **Java + Gradle** para a execução dos testes automatizados.
- API disponível em <http://localhost:8080>.
- Base de dados **H2**, criada automaticamente pela aplicação.

Os testes foram executados através do comando:

Código

```
./gradlew test
```

Este ambiente simples e controlado permitiu reproduzir facilmente os cenários de teste e identificar comportamentos inconsistentes.

5. Casos de Teste (em anexo)

Os casos de teste definidos para este projeto foram especificados seguindo rigorosamente o modelo apresentado no enunciado da unidade curricular, garantindo que cada cenário fosse descrito de forma completa, clara e verificável. Para assegurar uma melhor organização e legibilidade, **todos os casos de teste (CT001–CT053)** serão entregues em **três ficheiros XLS anexos**, correspondentes às entidades **Book**, **Member** e **Reservation**.

Cada tabela inclui os seguintes **campos obrigatórios**, conforme o exemplo fornecido no enunciado:

- **Nome**
- **Descrição**
- **Prioridade**
- **Condições iniciais**
- **Procedimento**
- **Dados de entrada**
- **Resultado esperado**
- **Sucesso**



- **Técnica**

Adicionalmente, e devido às particularidades observadas durante o processo de teste, foi incluído um **campo opcional de Observações**, utilizado apenas nos casos em que foram detetadas **inconsistências entre o comportamento real da API e a documentação Swagger**. Estas inconsistências não ocorreram de forma transversal, mas sim **em testes específicos**, sobretudo em cenários negativos ou em operações onde a API não implementava totalmente o comportamento documentado.

A opção por anexar os casos de teste em ficheiros XLS permite:

- maior clareza visual
- filtragem e ordenação por prioridade, entidade ou resultado
- separação lógica entre entidades
- evitar sobrecarga visual no relatório principal

6. Execução e Resultados Obtidos

A execução dos testes foi realizada através do comando:

Código

```
./gradlew test
```

Durante este processo, foram observados comportamentos relevantes que permitiram avaliar a maturidade da API. Embora a maioria dos endpoints funcionasse conforme esperado, verificaram-se **inconsistências pontuais** entre a documentação Swagger e o comportamento real da API, nomeadamente:

- códigos de estado divergentes em cenários negativos
- validações incompletas em campos obrigatórios
- ausência de endpoints documentados (ex.: remoção de reservas)
- respostas incoerentes para IDs inexistentes

Estas situações foram devidamente registadas no campo **Observações** das tabelas anexas, permitindo uma análise clara e contextualizada de cada ocorrência.

Os resultados completos encontram-se nos ficheiros XLS anexos, incluindo o estado final de cada teste (Passou / Falhou) e o comportamento observado.



7. Limpeza de Dados e Repetibilidade

Para garantir que os testes pudessem ser executados de forma repetível e independente, foi definida uma estratégia de limpeza de dados que incluiu:

- remoção de livros e membros através de DELETE com forceRemove=true
- remoção indireta de reservas ao eliminar entidades associadas
- reinicialização total da base de dados através da eliminação da pasta tmp
- reinício da API com java -jar library.jar

Esta abordagem assegurou que cada execução começasse num estado consistente, evitando interferências entre testes.

8. Discussão / Avaliação dos Resultados

A análise dos resultados evidencia que a API Library, embora funcional nos seus aspetos essenciais, apresenta limitações que comprometem a sua consistência e previsibilidade. As principais fragilidades identificadas incluem:

- **inconsistência nos códigos HTTP**, sobretudo em cenários negativos
- **validações insuficientes**, permitindo dados incompletos ou inválidos
- **divergências entre Swagger e implementação**, afetando a fiabilidade da documentação
- **ausência de operações fundamentais**, como a remoção de reservas
- **comportamentos inesperados** em operações de atualização e consulta

Apesar destas limitações, o processo de teste permitiu identificar claramente os pontos críticos e fornecer uma base sólida para futuras melhorias da API.

9. Conclusão

O trabalho desenvolvido permitiu aplicar de forma prática os conceitos abordados na unidade curricular de Testes de Software, desde a definição de cenários e casos de



Universidade de Aveiro

90338-TS

Testes de Software

teste até à execução, análise e discussão dos resultados. A API demonstrou comportamentos corretos em vários cenários, mas também revelou falhas estruturais que foram identificadas e documentadas.

A utilização de testes automatizados, aliada a uma estratégia de limpeza consistente, garantiu a repetibilidade e fiabilidade do processo. Os ficheiros anexos complementam este relatório, fornecendo uma visão detalhada do comportamento da API em cada cenário testado.